

Image Processing

Laboratory Manual

Course Code: CSC321

Semester V

Practical Sessions (10 Hours)

Prepared by:

Kiran Bagale

Department of Computer Science

December, 2025

Contents

Introduction	4
1 Practical Session 1: Introduction to Digital Images (1.5 hours)	5
1.1 Objectives	5
1.2 Theory Overview	5
1.3 Exercise 1.1: Loading and Displaying Images (20 min)	5
1.4 Exercise 1.2: Creating Images from Scratch (20 min)	6
1.5 Exercise 1.3: Sampling and Quantization (30 min)	7
1.6 Exercise 1.4: Neighborhood and Connectivity (20 min)	8
1.7 Assignment	9
2 Practical Session 2: Spatial Domain Image Enhancement (1.5 hours)	10
2.1 Objectives	10
2.2 Exercise 2.1: Gray Level Transformations (25 min)	10
2.3 Exercise 2.2: Bit Plane Slicing (20 min)	11
2.4 Exercise 2.3: Histogram Processing (25 min)	12
2.5 Exercise 2.4: Histogram Matching (20 min)	14
2.6 Assignment	15
3 Practical Session 3: Spatial Filtering (1.5 hours)	16
3.1 Objectives	16
3.2 Exercise 3.1: Basic Spatial Filtering (25 min)	16
3.3 Exercise 3.2: Non-linear Filters (25 min)	17
3.4 Exercise 3.3: Sharpening Filters (25 min)	18
3.5 Exercise 3.4: Unsharp Masking (15 min)	20
3.6 Assignment	21
4 Practical Session 4: Frequency Domain Processing (1.5 hours)	22
4.1 Objectives	22
4.2 Exercise 4.1: Fourier Transform Basics (25 min)	22
4.3 Exercise 4.2: Frequency Domain Filters (30 min)	23
4.4 Exercise 4.3: Band-pass and Band-reject Filters (20 min)	25
4.5 Exercise 4.4: Other Image Transforms (15 min)	26
4.6 Assignment	27
5 Practical Session 5: Image Restoration and Compression (1.5 hours)	28
5.1 Objectives	28
5.2 Exercise 5.1: Noise Models (20 min)	28
5.3 Exercise 5.2: Restoration Filters (25 min)	29
5.4 Exercise 5.3: Huffman Coding (25 min)	31
5.5 Exercise 5.4: Run-Length Encoding (20 min)	34
5.6 Assignment	35
6 Practical Session 6: Morphological Image Processing (1 hour)	36
6.1 Objectives	36
6.2 Exercise 6.1: Basic Morphological Operations (30 min)	36
6.3 Exercise 6.2: Advanced Morphological Operations (30 min)	38

6.4	Assignment	40
7	Practical Session 7: Image Segmentation (1.5 hours)	41
7.1	Objectives	41
7.2	Exercise 7.1: Edge Detection (25 min)	41
7.3	Exercise 7.2: Thresholding Techniques (25 min)	42
7.4	Exercise 7.3: Region-Based Segmentation (30 min)	44
7.5	Exercise 7.4: Hough Transform (20 min)	46
7.6	Assignment	48
7.7	Practical Session: Python Implementation	48
7.7.1	1. Split and Merge Algorithm for Region-Based Segmentation . .	48
7.7.2	2. Comparison of Edge Detection Methods	49
7.7.3	Observation	50

Introduction

This laboratory manual contains practical exercises for the Image Processing course (CSC321). The sessions are designed to provide hands-on experience with fundamental concepts and techniques in digital image processing. Students will implement various algorithms using Python with libraries such as NumPy, OpenCV, Matplotlib, and scikit-image.

Prerequisites

- Basic knowledge of Python programming
- Understanding of matrix operations
- Familiarity with NumPy and Matplotlib libraries

Software Requirements

- Python 3.8 or higher
- Libraries: NumPy, OpenCV (cv2), Matplotlib, scikit-image, scipy
- Jupyter Notebook (optional but recommended)

Installation Commands

```
1 pip install numpy opencv-python matplotlib scikit-image scipy
```

1 Practical Session 1: Introduction to Digital Images (1.5 hours)

1.1 Objectives

- Understand digital image representation
- Work with pixels and image matrices
- Perform basic image operations
- Understand sampling and quantization

1.2 Theory Overview

A digital image is a 2D function $f(x, y)$ where x and y are spatial coordinates, and the value of f at any point represents the intensity or gray level. In matrix form, an image can be represented as:

$$I = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

1.3 Exercise 1.1: Loading and Displaying Images (20 min)

Task

Load an image, display it, and examine its properties (dimensions, data type, pixel values).

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Load image in grayscale
6 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
7
8 # Display image properties
9 print(f"Image_shape: {img.shape}")
10 print(f"Image_data_type: {img.dtype}")
11 print(f"Min_pixel_value: {img.min()}")
12 print(f"Max_pixel_value: {img.max()}")
13
14 # Display the image
15 plt.figure(figsize=(8, 6))
16 plt.imshow(img, cmap='gray')
17 plt.title('Original Image')
18 plt.colorbar()
19 plt.axis('off')
```

```

20 plt.show()
21
22 # Access and print specific pixel values
23 print(f"Pixel at (100, 100): {img[100, 100]}")

```

Listing 1: Loading and displaying images

1.4 Exercise 1.2: Creating Images from Scratch (20 min)

Task

Create simple images using NumPy arrays and understand pixel manipulation.

```

1 # Create a black image
2 black_img = np.zeros((300, 300), dtype=np.uint8)
3
4 # Create a white image
5 white_img = np.ones((300, 300), dtype=np.uint8) * 255
6
7 # Create a gradient image
8 gradient = np.linspace(0, 255, 300, dtype=np.uint8)
9 gradient_img = np.tile(gradient, (300, 1))
10
11 # Create a checkerboard pattern
12 checkerboard = np.zeros((300, 300), dtype=np.uint8)
13 square_size = 30
14 for i in range(0, 300, square_size):
15     for j in range(0, 300, square_size):
16         if ((i // square_size) + (j // square_size)) % 2 == 0:
17             checkerboard[i:i+square_size, j:j+square_size] = 255
18
19 # Display all images
20 fig, axes = plt.subplots(2, 2, figsize=(10, 10))
21 axes[0, 0].imshow(black_img, cmap='gray')
22 axes[0, 0].set_title('Black Image')
23 axes[0, 1].imshow(white_img, cmap='gray')
24 axes[0, 1].set_title('White Image')
25 axes[1, 0].imshow(gradient_img, cmap='gray')
26 axes[1, 0].set_title('Gradient Image')
27 axes[1, 1].imshow(checkerboard, cmap='gray')
28 axes[1, 1].set_title('Checkerboard Pattern')
29 plt.tight_layout()
30 plt.show()

```

Listing 2: Creating images programmatically

1.5 Exercise 1.3: Sampling and Quantization (30 min)

Task

Implement downsampling and quantization to understand spatial and gray-level resolution.

```

1 def downsample_image(img, factor):
2     """Downsample image by given factor"""
3     return img[::factor, ::factor]
4
5 def quantize_image(img, levels):
6     """Quantize image to specified number of gray levels"""
7     factor = 256 // levels
8     quantized = (img // factor) * factor
9     return quantized.astype(np.uint8)
10
11 # Load original image
12 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
13
14 # Create different sampling versions
15 downsample_2 = downsample_image(img, 2)
16 downsample_4 = downsample_image(img, 4)
17 downsample_8 = downsample_image(img, 8)
18
19 # Create different quantization versions
20 quantize_128 = quantize_image(img, 128)
21 quantize_32 = quantize_image(img, 32)
22 quantize_8 = quantize_image(img, 8)
23 quantize_2 = quantize_image(img, 2)
24
25 # Display sampling results
26 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
27 axes[0, 0].imshow(img, cmap='gray')
28 axes[0, 0].set_title('Original')
29 axes[0, 1].imshow(downsample_2, cmap='gray')
30 axes[0, 1].set_title('Downsampled by 2')
31 axes[1, 0].imshow(downsample_4, cmap='gray')
32 axes[1, 0].set_title('Downsampled by 4')
33 axes[1, 1].imshow(downsample_8, cmap='gray')
34 axes[1, 1].set_title('Downsampled by 8')
35 plt.tight_layout()
36 plt.show()
37
38 # Display quantization results
39 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
40 axes[0, 0].imshow(quantize_128, cmap='gray')
41 axes[0, 0].set_title('128 Gray Levels')
42 axes[0, 1].imshow(quantize_32, cmap='gray')
43 axes[0, 1].set_title('32 Gray Levels')
44 axes[1, 0].imshow(quantize_8, cmap='gray')

```

```

45 axes[1, 0].set_title('8_Gray_Levels')
46 axes[1, 1].imshow(quantize_2, cmap='gray')
47 axes[1, 1].set_title('2_Gray_Levels(Binary)')
48 plt.tight_layout()
49 plt.show()

```

Listing 3: Sampling and quantization demonstration

1.6 Exercise 1.4: Neighborhood and Connectivity (20 min)

Task

Implement functions to find neighbors of a pixel and calculate distances.

```

1 def get_4_neighbors(img, x, y):
2     """Get_4-connected_neighbors_of_pixel(x,y)"""
3     neighbors = []
4     h, w = img.shape
5
6     if x > 0: neighbors.append((x-1, y))
7     if x < h-1: neighbors.append((x+1, y))
8     if y > 0: neighbors.append((x, y-1))
9     if y < w-1: neighbors.append((x, y+1))
10
11     return neighbors
12
13 def get_8_neighbors(img, x, y):
14     """Get_8-connected_neighbors_of_pixel(x,y)"""
15     neighbors = []
16     h, w = img.shape
17
18     for i in range(-1, 2):
19         for j in range(-1, 2):
20             if i == 0 and j == 0:
21                 continue
22             nx, ny = x + i, y + j
23             if 0 <= nx < h and 0 <= ny < w:
24                 neighbors.append((nx, ny))
25
26     return neighbors
27
28 def euclidean_distance(p1, p2):
29     """Calculate_Euclidean_distance_between_two_pixels"""
30     return np.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)
31
32 def manhattan_distance(p1, p2):
33     """Calculate_Manhattan_distance_between_two_pixels"""
34     return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
35
36 def chessboard_distance(p1, p2):
37     """Calculate_Chessboard_distance_between_two_pixels"""

```

```
38     return max(abs(p1[0] - p2[0]), abs(p1[1] - p2[1]))
39
40 # Example usage
41 img = np.zeros((100, 100), dtype=np.uint8)
42 pixel = (50, 50)
43
44 neighbors_4 = get_4_neighbors(img, *pixel)
45 neighbors_8 = get_8_neighbors(img, *pixel)
46
47 print(f"4-connected_neighbors_of_{pixel}:_{neighbors_4}")
48 print(f"8-connected_neighbors_of_{pixel}:_{neighbors_8}")
49
50 # Calculate distances
51 p1, p2 = (10, 10), (30, 40)
52 print(f"\nDistances_between_{p1}_and_{p2}:")
53 print(f"Euclidean:_{euclidean_distance(p1, p2):.2f}")
54 print(f"Manhattan:_{manhattan_distance(p1, p2)}")
55 print(f"Chessboard:_{chessboard_distance(p1, p2)}")
```

Listing 4: Pixel neighborhood and distance measures

1.7 Assignment

1. Load a color image and separate it into R, G, B channels. Display each channel separately.
2. Create a function to calculate the file size of an image based on dimensions and bit depth.
3. Implement a function to create a circular mask and apply it to an image.

2 Practical Session 2: Spatial Domain Image Enhancement (1.5 hours)

2.1 Objectives

- Implement point operations for image enhancement
- Apply histogram processing techniques
- Understand contrast stretching and intensity transformations

2.2 Exercise 2.1: Gray Level Transformations (25 min)

Task

Implement various intensity transformation functions.

```

1 def negative_transform(img):
2     """Create negative of image"""
3     return 255 - img
4
5 def log_transform(img, c=1):
6     """Apply logarithmic transformation"""
7     img_float = img.astype(np.float32)
8     log_img = c * np.log(1 + img_float)
9     log_img = np.uint8(255 * log_img / np.max(log_img))
10    return log_img
11
12 def power_law_transform(img, gamma):
13     """Apply power-law (gamma) transformation"""
14     img_normalized = img / 255.0
15     gamma_corrected = np.power(img_normalized, gamma)
16     return np.uint8(gamma_corrected * 255)
17
18 def contrast_stretching(img, r1, s1, r2, s2):
19     """Apply contrast stretching"""
20     output = np.zeros_like(img, dtype=np.float32)
21
22     # Below r1
23     mask1 = img < r1
24     output[mask1] = (s1 / r1) * img[mask1]
25
26     # Between r1 and r2
27     mask2 = (img >= r1) & (img <= r2)
28     output[mask2] = ((s2 - s1) / (r2 - r1)) * (img[mask2] - r1) +
29         s1
30
31     # Above r2
32     mask3 = img > r2
33     output[mask3] = ((255 - s2) / (255 - r2)) * (img[mask3] - r2)
34         + s2

```

```

33
34     return np.uint8(output)
35
36 def intensity_level_slicing(img, range_min, range_max,
37                             highlight_value=255):
38     """Slice specific intensity levels"""
39     output = np.zeros_like(img)
40     mask = (img >= range_min) & (img <= range_max)
41     output[mask] = highlight_value
42     return output
43
44 # Load and apply transformations
45 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
46
47 negative = negative_transform(img)
48 log_trans = log_transform(img)
49 gamma_05 = power_law_transform(img, 0.5)
50 gamma_15 = power_law_transform(img, 1.5)
51 contrast = contrast_stretching(img, 70, 0, 180, 255)
52 sliced = intensity_level_slicing(img, 100, 150)
53
54 # Display results
55 fig, axes = plt.subplots(2, 4, figsize=(16, 8))
56 images = [img, negative, log_trans, gamma_05,
57           gamma_15, contrast, sliced]
58 titles = ['Original', 'Negative', 'Log Transform',
59           'Gamma=0.5', 'Gamma=1.5', 'Contrast Stretch',
60           'Intensity Slicing']
61
62 for ax, image, title in zip(axes.flat, images, titles):
63     ax.imshow(image, cmap='gray')
64     ax.set_title(title)
65     ax.axis('off')
66
67 plt.tight_layout()
68 plt.show()

```

Listing 5: Gray level transformations

2.3 Exercise 2.2: Bit Plane Slicing (20 min)

Task

Extract and display individual bit planes of an image.

```

1 def extract_bit_plane(img, bit):
2     """Extract specific bit plane (0-7)"""
3     return ((img >> bit) & 1) * 255
4
5 # Load image
6 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)

```

```

7
8 # Extract all 8 bit planes
9 bit_planes = []
10 for i in range(8):
11     bit_planes.append(extract_bit_plane(img, i))
12
13 # Display all bit planes
14 fig, axes = plt.subplots(3, 3, figsize=(12, 12))
15 axes[0, 0].imshow(img, cmap='gray')
16 axes[0, 0].set_title('Original Image')
17 axes[0, 0].axis('off')
18
19 for i in range(8):
20     row = (i + 1) // 3
21     col = (i + 1) % 3
22     axes[row, col].imshow(bit_planes[i], cmap='gray')
23     axes[row, col].set_title(f'Bit Plane {i}')
24     axes[row, col].axis('off')
25
26 plt.tight_layout()
27 plt.show()
28
29 # Reconstruct image from bit planes
30 reconstructed = np.zeros_like(img)
31 for i in range(8):
32     reconstructed += (bit_planes[i] // 255) << i
33
34 print(f"Reconstruction error: {np.sum(np.abs(img - reconstructed))}")

```

Listing 6: Bit plane slicing

2.4 Exercise 2.3: Histogram Processing (25 min)

Task

Compute and display image histograms, implement histogram equalization.

```

1 def compute_histogram(img):
2     """Compute normalized histogram"""
3     hist, bins = np.histogram(img.flatten(), 256, [0, 256])
4     return hist
5
6 def histogram_equalization(img):
7     """Perform histogram equalization"""
8     # Compute histogram
9     hist = compute_histogram(img)
10
11     # Compute CDF
12     cdf = hist.cumsum()
13     cdf_normalized = cdf * 255 / cdf[-1]

```

```

14
15     # Use CDF as lookup table
16     equalized = np.interp(img.flatten(), range(256),
17                             cdf_normalized)
18
19     return equalized.reshape(img.shape).astype(np.uint8)
20
21
22 # Load image
23 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
24
25 # Apply histogram equalization
26 img_eq = histogram_equalization(img)
27
28
29 # Compute histograms
30 hist_orig = compute_histogram(img)
31 hist_eq = compute_histogram(img_eq)
32
33
34 # Display results
35 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
36
37 axes[0, 0].imshow(img, cmap='gray')
38 axes[0, 0].set_title('Original Image')
39 axes[0, 0].axis('off')
40
41 axes[0, 1].imshow(img_eq, cmap='gray')
42 axes[0, 1].set_title('Equalized Image')
43 axes[0, 1].axis('off')
44
45 axes[1, 0].bar(range(256), hist_orig)
46 axes[1, 0].set_title('Original Histogram')
47 axes[1, 0].set_xlim([0, 255])
48
49 axes[1, 1].bar(range(256), hist_eq)
50 axes[1, 1].set_title('Equalized Histogram')
51 axes[1, 1].set_xlim([0, 255])
52
53 plt.tight_layout()
54 plt.show()
55
56
57 # Compare statistics
58 print("Original Image Statistics:")
59 print(f"Mean: {np.mean(img):.2f}")
60 print(f"Std Dev: {np.std(img):.2f}")
61 print(f"Min: {np.min(img)}, Max: {np.max(img)}")
62
63 print("\nEqualized Image Statistics:")
64 print(f"Mean: {np.mean(img_eq):.2f}")
65 print(f"Std Dev: {np.std(img_eq):.2f}")
66 print(f"Min: {np.min(img_eq)}, Max: {np.max(img_eq)}")

```

Listing 7: Histogram processing and equalization

2.5 Exercise 2.4: Histogram Matching (20 min)

Task

Implement histogram matching to match one image's histogram to another.

```

1 def histogram_matching(source, reference):
2     """Match histogram of source to reference image"""
3     # Compute histograms
4     src_hist = compute_histogram(source)
5     ref_hist = compute_histogram(reference)
6
7     # Compute CDFs
8     src_cdf = src_hist.cumsum()
9     src_cdf = src_cdf / src_cdf[-1]
10
11     ref_cdf = ref_hist.cumsum()
12     ref_cdf = ref_cdf / ref_cdf[-1]
13
14     # Create lookup table
15     lookup = np.zeros(256, dtype=np.uint8)
16     for i in range(256):
17         j = np.argmin(np.abs(ref_cdf - src_cdf[i]))
18         lookup[i] = j
19
20     # Apply lookup table
21     matched = lookup[source]
22     return matched
23
24 # Load images
25 source = cv2.imread('source_image.jpg', cv2.IMREAD_GRAYSCALE)
26 reference = cv2.imread('reference_image.jpg', cv2.
27     IMREAD_GRAYSCALE)
28
29 # Perform histogram matching
30 matched = histogram_matching(source, reference)
31
32 # Display results
33 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
34
35 axes[0, 0].imshow(source, cmap='gray')
36 axes[0, 0].set_title('Source Image')
37 axes[0, 1].imshow(reference, cmap='gray')
38 axes[0, 1].set_title('Reference Image')
39 axes[0, 2].imshow(matched, cmap='gray')
40 axes[0, 2].set_title('Matched Image')
41
42 axes[1, 0].hist(source.flatten(), 256, [0, 256])
43 axes[1, 0].set_title('Source Histogram')
44 axes[1, 1].hist(reference.flatten(), 256, [0, 256])
45 axes[1, 1].set_title('Reference Histogram')

```

```
45 axes[1, 2].hist(matched.flatten(), 256, [0, 256])
46 axes[1, 2].set_title('Matched_Histogram')
47
48 plt.tight_layout()
49 plt.show()
```

Listing 8: Histogram matching

2.6 Assignment

1. Implement adaptive histogram equalization for local contrast enhancement.
2. Create a function for piecewise linear transformation with multiple control points.
3. Experiment with different gamma values and observe effects on dark and bright images.

3 Practical Session 3: Spatial Filtering (1.5 hours)

3.1 Objectives

- Understand convolution and correlation
- Implement linear and non-linear spatial filters
- Apply smoothing and sharpening filters

3.2 Exercise 3.1: Basic Spatial Filtering (25 min)

Task

Implement convolution operation and apply basic averaging filters.

```

1 from scipy import signal
2
3 def apply_filter(img, kernel):
4     """Apply a filter kernel to image using convolution"""
5     return signal.convolve2d(img, kernel, mode='same',
6                               boundary='symm')
7
8 # Load image
9 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
10
11 # Define various kernels
12 # Average filter (3x3)
13 kernel_avg_3 = np.ones((3, 3)) / 9
14
15 # Average filter (5x5)
16 kernel_avg_5 = np.ones((5, 5)) / 25
17
18 # Weighted average filter
19 kernel_weighted = np.array([[1, 2, 1],
20                               [2, 4, 2],
21                               [1, 2, 1]]) / 16
22
23 # Apply filters
24 img_avg_3 = apply_filter(img, kernel_avg_3)
25 img_avg_5 = apply_filter(img, kernel_avg_5)
26 img_weighted = apply_filter(img, kernel_weighted)
27
28 # Display results
29 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
30
31 axes[0, 0].imshow(img, cmap='gray')
32 axes[0, 0].set_title('Original Image')
33 axes[0, 1].imshow(img_avg_3, cmap='gray')
34 axes[0, 1].set_title('3x3 Average Filter')
35 axes[1, 0].imshow(img_avg_5, cmap='gray')

```

```

36 axes[1, 0].set_title('5x5_Average_Filter')
37 axes[1, 1].imshow(img_weighted, cmap='gray')
38 axes[1, 1].set_title('Weighted_Average_Filter')
39
40 for ax in axes.flat:
41     ax.axis('off')
42
43 plt.tight_layout()
44 plt.show()

```

Listing 9: Spatial filtering implementation

3.3 Exercise 3.2: Non-linear Filters (25 min)

Task

Implement order-statistic filters including median, min, max, and midpoint filters.

```

1 from scipy.ndimage import median_filter, minimum_filter, \
2     maximum_filter
3
4 def midpoint_filter(img, size):
5     """Apply midpoint filter"""
6     min_filtered = minimum_filter(img, size=size)
7     max_filtered = maximum_filter(img, size=size)
8     return ((min_filtered.astype(np.float32) +
9             max_filtered.astype(np.float32)) / 2).astype(np.
10                uint8)
11
12 def alpha_trimmed_mean(img, size, d):
13     """Apply alpha-trimmed mean filter
14     d: number of values to trim from each end"""
15     from scipy.ndimage import generic_filter
16
17     def trimmed_mean(values):
18         sorted_vals = np.sort(values)
19         trimmed = sorted_vals[d:-d] if d > 0 else sorted_vals
20         return np.mean(trimmed)
21
22     return generic_filter(img, trimmed_mean, size=size)
23
24 # Load and add noise
25 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
26
27 # Add salt and pepper noise
28 noise_img = img.copy()
29 num_salt = np.ceil(0.05 * img.size)
30 coords = [np.random.randint(0, i - 1, int(num_salt))
31           for i in img.shape]
32 noise_img[coords[0], coords[1]] = 255

```

```

33 num_pepper = np.ceil(0.05 * img.size)
34 coords = [np.random.randint(0, i - 1, int(num_pepper))
35            for i in img.shape]
36 noise_img[coords[0], coords[1]] = 0
37
38 # Apply filters
39 median_filtered = median_filter(noise_img, size=3)
40 min_filtered = minimum_filter(noise_img, size=3)
41 max_filtered = maximum_filter(noise_img, size=3)
42 midpoint_filtered = midpoint_filter(noise_img, 3)
43 alpha_trimmed = alpha_trimmed_mean(noise_img, 5, 2)
44
45 # Display results
46 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
47
48 axes[0, 0].imshow(img, cmap='gray')
49 axes[0, 0].set_title('Original')
50 axes[0, 1].imshow(noise_img, cmap='gray')
51 axes[0, 1].set_title('Noisy Image')
52 axes[0, 2].imshow(median_filtered, cmap='gray')
53 axes[0, 2].set_title('Median Filter')
54 axes[1, 0].imshow(min_filtered, cmap='gray')
55 axes[1, 0].set_title('Min Filter')
56 axes[1, 1].imshow(max_filtered, cmap='gray')
57 axes[1, 1].set_title('Max Filter')
58 axes[1, 2].imshow(alpha_trimmed, cmap='gray')
59 axes[1, 2].set_title('Alpha-Trimmed Mean')
60
61 for ax in axes.flat:
62     ax.axis('off')
63
64 plt.tight_layout()
65 plt.show()

```

Listing 10: Non-linear filtering

3.4 Exercise 3.3: Sharpening Filters (25 min)

Task

Implement gradient-based and Laplacian sharpening filters.

```

1 def apply_laplacian(img, kernel_type='basic'):
2     """Apply Laplacian filter"""
3     if kernel_type == 'basic':
4         kernel = np.array([[0, 1, 0],
5                             [1, -4, 1],
6                             [0, 1, 0]])
7     elif kernel_type == 'diagonal':
8         kernel = np.array([[1, 1, 1],
9                             [1, -8, 1],

```

```

10         [1, 1, 1]])
11
12     laplacian = apply_filter(img, kernel)
13     # Sharpen by subtracting Laplacian
14     sharpened = img - laplacian
15     return np.clip(sharpened, 0, 255).astype(np.uint8)
16
17 def apply_high_boost(img, kernel, k=1):
18     """Apply high-boost filter"""
19     blurred = apply_filter(img, kernel)
20     mask = img - blurred
21     boosted = img + k * mask
22     return np.clip(boosted, 0, 255).astype(np.uint8)
23
24 # Roberts Cross Gradient
25 roberts_x = np.array([[1, 0],
26                       [0, -1]])
27 roberts_y = np.array([[0, 1],
28                       [-1, 0]])
29
30 # Prewitt filters
31 prewitt_x = np.array([[-1, 0, 1],
32                       [-1, 0, 1],
33                       [-1, 0, 1]])
34 prewitt_y = np.array([[-1, -1, -1],
35                       [0, 0, 0],
36                       [1, 1, 1]])
37
38 # Sobel filters
39 sobel_x = np.array([[-1, 0, 1],
40                     [-2, 0, 2],
41                     [-1, 0, 1]])
42 sobel_y = np.array([[-1, -2, -1],
43                     [0, 0, 0],
44                     [1, 2, 1]])
45
46 # Load image
47 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
48
49 # Apply gradient filters
50 roberts = np.sqrt(apply_filter(img, roberts_x)**2 +
51                  apply_filter(img, roberts_y)**2)
52 prewitt = np.sqrt(apply_filter(img, prewitt_x)**2 +
53                  apply_filter(img, prewitt_y)**2)
54 sobel = np.sqrt(apply_filter(img, sobel_x)**2 +
55                apply_filter(img, sobel_y)**2)
56
57 # Apply Laplacian
58 laplacian_sharp = apply_laplacian(img, 'basic')
59
60 # Apply high-boost filter

```

```

61 avg_kernel = np.ones((3, 3)) / 9
62 high_boost = apply_high_boost(img, avg_kernel, k=1.5)
63
64 # Display results
65 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
66
67 axes[0, 0].imshow(img, cmap='gray')
68 axes[0, 0].set_title('Original')
69 axes[0, 1].imshow(roberts, cmap='gray')
70 axes[0, 1].set_title('Roberts')
71 axes[0, 2].imshow(rewitt, cmap='gray')
72 axes[0, 2].set_title('Prewitt')
73 axes[1, 0].imshow(sobel, cmap='gray')
74 axes[1, 0].set_title('Sobel')
75 axes[1, 1].imshow(laplacian_sharp, cmap='gray')
76 axes[1, 1].set_title('Laplacian Sharpening')
77 axes[1, 2].imshow(high_boost, cmap='gray')
78 axes[1, 2].set_title('High-Boost Filter')
79
80 for ax in axes.flat:
81     ax.axis('off')
82
83 plt.tight_layout()
84 plt.show()

```

Listing 11: Sharpening filters

3.5 Exercise 3.4: Unsharp Masking (15 min)

Task

Implement unsharp masking for image sharpening.

```

1 def unsharp_mask(img, sigma=1.0, amount=1.5, threshold=0):
2     """Apply unsharp masking"""
3     from scipy.ndimage import gaussian_filter
4
5     # Blur the image
6     blurred = gaussian_filter(img.astype(float), sigma)
7
8     # Calculate the mask (difference)
9     mask = img - blurred
10
11    # Apply threshold
12    if threshold > 0:
13        low_contrast_mask = np.absolute(mask) < threshold
14        mask[low_contrast_mask] = 0
15
16    # Add weighted mask to original
17    sharpened = img + amount * mask
18

```

```
19     return np.clip(sharpened, 0, 255).astype(np.uint8)
20
21 # Load image
22 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
23
24 # Apply unsharp masking with different parameters
25 unsharp_1 = unsharp_mask(img, sigma=1.0, amount=1.0)
26 unsharp_2 = unsharp_mask(img, sigma=1.0, amount=2.0)
27 unsharp_3 = unsharp_mask(img, sigma=2.0, amount=1.5)
28
29 # Display results
30 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
31
32 axes[0, 0].imshow(img, cmap='gray')
33 axes[0, 0].set_title('Original')
34 axes[0, 1].imshow(unsharp_1, cmap='gray')
35 axes[0, 1].set_title('Unsharp_1(sigma=1.0, amount=1.0)')
36 axes[1, 0].imshow(unsharp_2, cmap='gray')
37 axes[1, 0].set_title('Unsharp_2(sigma=1.0, amount=2.0)')
38 axes[1, 1].imshow(unsharp_3, cmap='gray')
39 axes[1, 1].set_title('Unsharp_3(sigma=2.0, amount=1.5)')
40
41 for ax in axes.flat:
42     ax.axis('off')
43
44 plt.tight_layout()
45 plt.show()
```

Listing 12: Unsharp masking

3.6 Assignment

1. Implement and compare different noise removal filters on images with various noise types.
2. Create a GUI to interactively adjust filter parameters and observe results.
3. Compare the performance (execution time) of different filtering methods.

4 Practical Session 4: Frequency Domain Processing (1.5 hours)

4.1 Objectives

- Understand Fourier Transform and frequency domain
- Implement FFT and inverse FFT
- Apply frequency domain filters

4.2 Exercise 4.1: Fourier Transform Basics (25 min)

Task

Compute and visualize the 2D Fourier Transform of images.

```
1 def visualize_fft(img):
2     """Compute and visualize FFT"""
3     # Compute FFT
4     f = np.fft.fft2(img)
5     fshift = np.fft.fftshift(f)
6
7     # Compute magnitude spectrum
8     magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)
9
10    # Compute phase spectrum
11    phase_spectrum = np.angle(fshift)
12
13    return fshift, magnitude_spectrum, phase_spectrum
14
15 def inverse_fft(fshift):
16     """Compute inverse FFT"""
17     f_ishift = np.fft.ifftshift(fshift)
18     img_back = np.fft.ifft2(f_ishift)
19     img_back = np.real(img_back)
20     return img_back
21
22 # Load image
23 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
24
25 # Compute FFT
26 fshift, magnitude, phase = visualize_fft(img)
27
28 # Reconstruct image
29 reconstructed = inverse_fft(fshift)
30
31 # Display results
32 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
33
34 axes[0, 0].imshow(img, cmap='gray')
```

```

35 axes[0, 0].set_title('Original_Image')
36 axes[0, 1].imshow(magnitude, cmap='gray')
37 axes[0, 1].set_title('Magnitude_Spectrum')
38 axes[1, 0].imshow(phase, cmap='gray')
39 axes[1, 0].set_title('Phase_Spectrum')
40 axes[1, 1].imshow(reconstructed, cmap='gray')
41 axes[1, 1].set_title('Reconstructed_Image')
42
43 for ax in axes.flat:
44     ax.axis('off')
45
46 plt.tight_layout()
47 plt.show()
48
49 # Verify reconstruction
50 error = np.sum(np.abs(img - reconstructed))
51 print(f"Reconstruction_error: {error:.2f}")

```

Listing 13: Fourier Transform implementation

4.3 Exercise 4.2: Frequency Domain Filters (30 min)

Task

Implement ideal, Butterworth, and Gaussian low-pass and high-pass filters in frequency domain.

```

1 def ideal_lowpass_filter(shape, cutoff):
2     """Create_ideal_low-pass_filter"""
3     rows, cols = shape
4     crow, ccol = rows // 2, cols // 2
5
6     mask = np.zeros((rows, cols), np.float32)
7     for i in range(rows):
8         for j in range(cols):
9             distance = np.sqrt((i - crow)**2 + (j - ccol)**2)
10            if distance <= cutoff:
11                mask[i, j] = 1
12    return mask
13
14 def butterworth_lowpass_filter(shape, cutoff, order=2):
15     """Create_Butterworth_low-pass_filter"""
16     rows, cols = shape
17     crow, ccol = rows // 2, cols // 2
18
19     u = np.arange(rows)
20     v = np.arange(cols)
21     u, v = np.meshgrid(u - crow, v - ccol, indexing='ij')
22
23     D = np.sqrt(u**2 + v**2)
24     mask = 1 / (1 + (D / cutoff)**(2 * order))

```

```

25     return mask
26
27 def gaussian_lowpass_filter(shape, cutoff):
28     """Create Gaussian low-pass filter"""
29     rows, cols = shape
30     crow, ccol = rows // 2, cols // 2
31
32     u = np.arange(rows)
33     v = np.arange(cols)
34     u, v = np.meshgrid(u - crow, v - ccol, indexing='ij')
35
36     D = np.sqrt(u**2 + v**2)
37     mask = np.exp(-(D**2) / (2 * (cutoff**2)))
38     return mask
39
40 def apply_frequency_filter(img, filter_mask):
41     """Apply frequency domain filter"""
42     # Compute FFT
43     f = np.fft.fft2(img)
44     fshift = np.fft.fftshift(f)
45
46     # Apply filter
47     filtered = fshift * filter_mask
48
49     # Inverse FFT
50     f_ishift = np.fft.ifftshift(filtered)
51     img_filtered = np.fft.ifft2(f_ishift)
52     img_filtered = np.real(img_filtered)
53
54     return np.clip(img_filtered, 0, 255).astype(np.uint8)
55
56 # Load image
57 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
58
59 # Create filters
60 cutoff = 30
61 ideal_lp = ideal_lowpass_filter(img.shape, cutoff)
62 butter_lp = butterworth_lowpass_filter(img.shape, cutoff, order
    =2)
63 gauss_lp = gaussian_lowpass_filter(img.shape, cutoff)
64
65 # High-pass filters (complement of low-pass)
66 ideal_hp = 1 - ideal_lp
67 butter_hp = 1 - butter_lp
68 gauss_hp = 1 - gauss_lp
69
70 # Apply low-pass filters
71 img_ideal_lp = apply_frequency_filter(img, ideal_lp)
72 img_butter_lp = apply_frequency_filter(img, butter_lp)
73 img_gauss_lp = apply_frequency_filter(img, gauss_lp)
74

```

```

75 # Apply high-pass filters
76 img_ideal_hp = apply_frequency_filter(img, ideal_hp)
77 img_butter_hp = apply_frequency_filter(img, butter_hp)
78 img_gauss_hp = apply_frequency_filter(img, gauss_hp)
79
80 # Display results
81 fig, axes = plt.subplots(3, 3, figsize=(15, 15))
82
83 axes[0, 0].imshow(img, cmap='gray')
84 axes[0, 0].set_title('Original')
85 axes[0, 1].imshow(ideal_lp, cmap='gray')
86 axes[0, 1].set_title('Ideal_LP_Filter')
87 axes[0, 2].imshow(img_ideal_lp, cmap='gray')
88 axes[0, 2].set_title('Ideal_LP_Result')
89
90 axes[1, 0].imshow(butter_lp, cmap='gray')
91 axes[1, 0].set_title('Butterworth_LP_Filter')
92 axes[1, 1].imshow(img_butter_lp, cmap='gray')
93 axes[1, 1].set_title('Butterworth_LP_Result')
94 axes[1, 2].imshow(img_ideal_hp, cmap='gray')
95 axes[1, 2].set_title('Ideal_HP_Result')
96
97 axes[2, 0].imshow(gauss_lp, cmap='gray')
98 axes[2, 0].set_title('Gaussian_LP_Filter')
99 axes[2, 1].imshow(img_gauss_lp, cmap='gray')
100 axes[2, 1].set_title('Gaussian_LP_Result')
101 axes[2, 2].imshow(img_gauss_hp, cmap='gray')
102 axes[2, 2].set_title('Gaussian_HP_Result')
103
104 for ax in axes.flat:
105     ax.axis('off')
106
107 plt.tight_layout()
108 plt.show()

```

Listing 14: Frequency domain filters

4.4 Exercise 4.3: Band-pass and Band-reject Filters (20 min)

Task

Implement band-pass and band-reject filters for selective frequency filtering.

```

1 def bandpass_filter(shape, inner_cutoff, outer_cutoff):
2     """Create band-pass filter"""
3     outer_filter = gaussian_lowpass_filter(shape, outer_cutoff)
4     inner_filter = gaussian_lowpass_filter(shape, inner_cutoff)
5     return outer_filter - inner_filter
6
7 def bandreject_filter(shape, inner_cutoff, outer_cutoff):
8     """Create band-reject filter"""

```

```

9         return 1 - bandpass_filter(shape, inner_cutoff, outer_cutoff)
10
11 # Load image
12 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
13
14 # Create band filters
15 bandpass = bandpass_filter(img.shape, 20, 60)
16 bandreject = bandreject_filter(img.shape, 20, 60)
17
18 # Apply filters
19 img_bandpass = apply_frequency_filter(img, bandpass)
20 img_bandreject = apply_frequency_filter(img, bandreject)
21
22 # Display results
23 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
24
25 axes[0, 0].imshow(img, cmap='gray')
26 axes[0, 0].set_title('Original Image')
27 axes[0, 1].imshow(bandpass, cmap='gray')
28 axes[0, 1].set_title('Band-pass Filter')
29 axes[1, 0].imshow(img_bandpass, cmap='gray')
30 axes[1, 0].set_title('Band-pass Result')
31 axes[1, 1].imshow(img_bandreject, cmap='gray')
32 axes[1, 1].set_title('Band-reject Result')
33
34 for ax in axes.flat:
35     ax.axis('off')
36
37 plt.tight_layout()
38 plt.show()

```

Listing 15: Band-pass and band-reject filters

4.5 Exercise 4.4: Other Image Transforms (15 min)

Task

Implement and apply Discrete Cosine Transform (DCT).

```

1 from scipy.fftpack import dct, idct
2
3 def apply_dct2(img):
4     """Apply 2D DCT"""
5     return dct(dct(img.T, norm='ortho').T, norm='ortho')
6
7 def apply_idct2(dct_img):
8     """Apply inverse 2D DCT"""
9     return idct(idct(dct_img.T, norm='ortho').T, norm='ortho')
10
11 # Load image
12 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)

```

```
13
14 # Apply DCT
15 dct_img = apply_dct2(img.astype(float))
16
17 # Visualize DCT coefficients (log scale)
18 dct_log = np.log(np.abs(dct_img) + 1)
19
20 # Reconstruct image
21 reconstructed = apply_idct2(dct_img)
22
23 # Display results
24 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
25
26 axes[0].imshow(img, cmap='gray')
27 axes[0].set_title('Original Image')
28 axes[1].imshow(dct_log, cmap='hot')
29 axes[1].set_title('DCT Coefficients (log scale)')
30 axes[2].imshow(reconstructed, cmap='gray')
31 axes[2].set_title('Reconstructed Image')
32
33 for ax in axes.flat:
34     ax.axis('off')
35
36 plt.tight_layout()
37 plt.show()
38
39 # Calculate reconstruction error
40 error = np.mean((img - reconstructed)**2)
41 print(f"MSE: {error:.4f}")
```

Listing 16: Discrete Cosine Transform

4.6 Assignment

1. Implement notch filters to remove periodic noise from images.
2. Compare the efficiency of DFT vs FFT for different image sizes.
3. Implement Hadamard and Haar transforms.

5 Practical Session 5: Image Restoration and Compression (1.5 hours)

5.1 Objectives

- Add and remove various types of noise
- Implement restoration filters
- Understand image compression techniques

5.2 Exercise 5.1: Noise Models (20 min)

Task

Implement and add various noise models to images.

```

1 def add_gaussian_noise(img, mean=0, sigma=25):
2     """Add Gaussian noise"""
3     noise = np.random.normal(mean, sigma, img.shape)
4     noisy = img + noise
5     return np.clip(noisy, 0, 255).astype(np.uint8)
6
7 def add_salt_pepper_noise(img, salt_prob=0.05, pepper_prob=0.05):
8     """Add salt and pepper noise"""
9     noisy = img.copy()
10
11     # Salt noise
12     num_salt = np.ceil(salt_prob * img.size)
13     coords = [np.random.randint(0, i, int(num_salt))
14               for i in img.shape]
15     noisy[coords[0], coords[1]] = 255
16
17     # Pepper noise
18     num_pepper = np.ceil(pepper_prob * img.size)
19     coords = [np.random.randint(0, i, int(num_pepper))
20               for i in img.shape]
21     noisy[coords[0], coords[1]] = 0
22
23     return noisy
24
25 def add_uniform_noise(img, low=-50, high=50):
26     """Add uniform noise"""
27     noise = np.random.uniform(low, high, img.shape)
28     noisy = img + noise
29     return np.clip(noisy, 0, 255).astype(np.uint8)
30
31 def add_rayleigh_noise(img, scale=50):
32     """Add Rayleigh noise"""
33     noise = np.random.rayleigh(scale, img.shape)
34     noisy = img + noise

```

```

35     return np.clip(noisy, 0, 255).astype(np.uint8)
36
37 def add_exponential_noise(img, scale=50):
38     """Add exponential noise"""
39     noise = np.random.exponential(scale, img.shape)
40     noisy = img + noise
41     return np.clip(noisy, 0, 255).astype(np.uint8)
42
43 # Load image
44 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
45
46 # Add different types of noise
47 gaussian_noisy = add_gaussian_noise(img, mean=0, sigma=25)
48 sp_noisy = add_salt_pepper_noise(img, 0.02, 0.02)
49 uniform_noisy = add_uniform_noise(img, -30, 30)
50 rayleigh_noisy = add_rayleigh_noise(img, 30)
51 exponential_noisy = add_exponential_noise(img, 30)
52
53 # Display results
54 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
55
56 axes[0, 0].imshow(img, cmap='gray')
57 axes[0, 0].set_title('Original')
58 axes[0, 1].imshow(gaussian_noisy, cmap='gray')
59 axes[0, 1].set_title('Gaussian Noise')
60 axes[0, 2].imshow(sp_noisy, cmap='gray')
61 axes[0, 2].set_title('Salt & Pepper Noise')
62 axes[1, 0].imshow(uniform_noisy, cmap='gray')
63 axes[1, 0].set_title('Uniform Noise')
64 axes[1, 1].imshow(rayleigh_noisy, cmap='gray')
65 axes[1, 1].set_title('Rayleigh Noise')
66 axes[1, 2].imshow(exponential_noisy, cmap='gray')
67 axes[1, 2].set_title('Exponential Noise')
68
69 for ax in axes.flat:
70     ax.axis('off')
71
72 plt.tight_layout()
73 plt.show()

```

Listing 17: Noise generation and addition

5.3 Exercise 5.2: Restoration Filters (25 min)

Task

Implement mean filters and order-statistic filters for noise removal.

```

1 from scipy.ndimage import generic_filter
2
3 def arithmetic_mean_filter(img, size):

```

```

4      """Arithmetic_mean_filter"""
5      kernel = np.ones((size, size)) / (size * size)
6      return apply_filter(img, kernel).astype(np.uint8)
7
8  def geometric_mean_filter(img, size):
9      """Geometric_mean_filter"""
10     def geom_mean(values):
11         return np.prod(values) ** (1.0 / len(values))
12
13     return generic_filter(img.astype(float), geom_mean,
14                           size=size).astype(np.uint8)
15
16  def harmonic_mean_filter(img, size):
17      """Harmonic_mean_filter"""
18     def harm_mean(values):
19         values = values[values > 0] # Avoid division by zero
20         if len(values) == 0:
21             return 0
22         return len(values) / np.sum(1.0 / values)
23
24     return generic_filter(img.astype(float), harm_mean,
25                           size=size).astype(np.uint8)
26
27  def contraharmonic_mean_filter(img, size, Q):
28      """Contraharmonic_mean_filter
29      Q>0: removes pepper noise
30      Q<0: removes salt noise"""
31     def contra_mean(values):
32         if Q == 0:
33             return np.mean(values)
34         numerator = np.sum(values ** (Q + 1))
35         denominator = np.sum(values ** Q)
36         if denominator == 0:
37             return 0
38         return numerator / denominator
39
40     return generic_filter(img.astype(float), contra_mean,
41                           size=size).astype(np.uint8)
42
43  # Create noisy images
44  img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
45  gaussian_noisy = add_gaussian_noise(img, 0, 20)
46  sp_noisy = add_salt_pepper_noise(img, 0.02, 0.02)
47
48  # Apply restoration filters to Gaussian noise
49  arith_mean = arithmetic_mean_filter(gaussian_noisy, 5)
50  geom_mean = geometric_mean_filter(gaussian_noisy, 5)
51  harm_mean = harmonic_mean_filter(gaussian_noisy, 5)
52
53  # Apply contraharmonic filter to salt & pepper noise
54  contra_salt = contraharmonic_mean_filter(sp_noisy, 5, -1.5)

```

```

55 contra_pepper = contraharmonic_mean_filter(sp_noisy, 5, 1.5)
56
57 # Display results for Gaussian noise
58 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
59
60 axes[0, 0].imshow(img, cmap='gray')
61 axes[0, 0].set_title('Original')
62 axes[0, 1].imshow(gaussian_noisy, cmap='gray')
63 axes[0, 1].set_title('Gaussian_Noisy')
64 axes[0, 2].imshow(arith_mean, cmap='gray')
65 axes[0, 2].set_title('Arithmetic_Mean')
66 axes[1, 0].imshow(geom_mean, cmap='gray')
67 axes[1, 0].set_title('Geometric_Mean')
68 axes[1, 1].imshow(harm_mean, cmap='gray')
69 axes[1, 1].set_title('Harmonic_Mean')
70 axes[1, 2].imshow(median_filter(gaussian_noisy, 5), cmap='gray')
71 axes[1, 2].set_title('Median_Filter')
72
73 for ax in axes.flat:
74     ax.axis('off')
75
76 plt.tight_layout()
77 plt.show()
78
79 # Display results for Salt & Pepper noise
80 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
81
82 axes[0].imshow(sp_noisy, cmap='gray')
83 axes[0].set_title('Salt_&_Pepper_Noisy')
84 axes[1].imshow(contra_salt, cmap='gray')
85 axes[1].set_title('Contraharmonic_(Q=-1.5)')
86 axes[2].imshow(contra_pepper, cmap='gray')
87 axes[2].set_title('Contraharmonic_(Q=1.5)')
88
89 for ax in axes:
90     ax.axis('off')
91
92 plt.tight_layout()
93 plt.show()

```

Listing 18: Restoration filters

5.4 Exercise 5.3: Huffman Coding (25 min)

Task

Implement Huffman coding for lossless image compression.

```

1 import heapq
2 from collections import defaultdict, Counter
3

```

```

4 class HuffmanNode:
5     def __init__(self, symbol=None, freq=0):
6         self.symbol = symbol
7         self.freq = freq
8         self.left = None
9         self.right = None
10
11     def __lt__(self, other):
12         return self.freq < other.freq
13
14 def build_huffman_tree(frequencies):
15     """Build Huffman tree from frequency dictionary"""
16     heap = [HuffmanNode(symbol, freq)
17             for symbol, freq in frequencies.items()]
18     heapq.heapify(heap)
19
20     while len(heap) > 1:
21         left = heapq.heappop(heap)
22         right = heapq.heappop(heap)
23
24         parent = HuffmanNode(freq=left.freq + right.freq)
25         parent.left = left
26         parent.right = right
27
28         heapq.heappush(heap, parent)
29
30     return heap[0]
31
32 def generate_codes(root, code="", codes=None):
33     """Generate Huffman codes from tree"""
34     if codes is None:
35         codes = {}
36
37     if root is None:
38         return codes
39
40     if root.symbol is not None:
41         codes[root.symbol] = code if code else "0"
42         return codes
43
44     generate_codes(root.left, code + "0", codes)
45     generate_codes(root.right, code + "1", codes)
46
47     return codes
48
49 def huffman_encode(data):
50     """Encode data using Huffman coding"""
51     # Calculate frequencies
52     frequencies = Counter(data.flatten())
53
54     # Build tree and generate codes

```

```

55     tree = build_huffman_tree(frequencies)
56     codes = generate_codes(tree)
57
58     # Encode data
59     encoded = ''.join(codes[pixel] for pixel in data.flatten())
60
61     return encoded, codes, frequencies
62
63 def calculate_compression_metrics(original_bits, encoded_bits):
64     """Calculate compression ratio and redundancy"""
65     compression_ratio = original_bits / encoded_bits
66     relative_redundancy = 1 - (1 / compression_ratio)
67
68     return compression_ratio, relative_redundancy
69
70 # Load image
71 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
72
73 # Reduce to smaller size for demonstration
74 img_small = img[0:100, 0:100]
75
76 # Perform Huffman encoding
77 encoded, codes, frequencies = huffman_encode(img_small)
78
79 # Calculate metrics
80 original_bits = img_small.size * 8 # 8 bits per pixel
81 encoded_bits = len(encoded)
82 comp_ratio, redundancy = calculate_compression_metrics(
83     original_bits, encoded_bits)
84
85 # Display Huffman codes
86 print("Huffman Codes (first 10 symbols):")
87 for i, (symbol, code) in enumerate(sorted(codes.items())[:10]):
88     print(f"Pixel value {symbol}: {code} (frequency: {frequencies[symbol]})")
89
90
91 print(f"\nOriginal size: {original_bits} bits")
92 print(f"Compressed size: {encoded_bits} bits")
93 print(f"Compression ratio: {comp_ratio:.2f}:1")
94 print(f"Relative data redundancy: {redundancy:.2%}")
95
96 # Calculate average code length
97 avg_length = sum(len(codes[s]) * frequencies[s]
98                 for s in codes) / sum(frequencies.values())
99 print(f"Average code length: {avg_length:.2f} bits/symbol")
100
101 # Plot frequency distribution
102 plt.figure(figsize=(12, 5))
103 plt.subplot(1, 2, 1)
104 plt.hist(img_small.flatten(), bins=256, range=(0, 256))
105 plt.title('Pixel Value Distribution')

```

```

106 plt.xlabel('Pixel_Value')
107 plt.ylabel('Frequency')
108
109 plt.subplot(1, 2, 2)
110 code_lengths = [len(codes[s]) for s in img_small.flatten()]
111 plt.hist(code_lengths, bins=max(code_lengths),
112          range=(0, max(code_lengths)))
113 plt.title('Code_Length_Distribution')
114 plt.xlabel('Code_Length(bits)')
115 plt.ylabel('Frequency')
116
117 plt.tight_layout()
118 plt.show()

```

Listing 19: Huffman coding implementation

5.5 Exercise 5.4: Run-Length Encoding (20 min)

Task

Implement Run-Length Encoding for image compression.

```

1 def run_length_encode(data):
2     """Encode data using run-length encoding"""
3     encoded = []
4     data_flat = data.flatten()
5
6     i = 0
7     while i < len(data_flat):
8         count = 1
9         value = data_flat[i]
10
11         while i + count < len(data_flat) and \
12               data_flat[i + count] == value and count < 255:
13             count += 1
14
15         encoded.append((value, count))
16         i += count
17
18     return encoded
19
20 def run_length_decode(encoded, shape):
21     """Decode run-length encoded data"""
22     decoded = []
23
24     for value, count in encoded:
25         decoded.extend([value] * count)
26
27     return np.array(decoded).reshape(shape)
28
29 # Create a simple binary image for demonstration

```

```

30 simple_img = np.zeros((100, 100), dtype=np.uint8)
31 simple_img[20:40, 20:80] = 255
32 simple_img[60:80, 30:70] = 255
33
34 # Encode
35 encoded = run_length_encode(simple_img)
36
37 # Decode
38 decoded = run_length_decode(encoded, simple_img.shape)
39
40 # Calculate compression
41 original_size = simple_img.size
42 compressed_size = len(encoded) * 2 # Each tuple is (value, count)
43 compression_ratio = original_size / compressed_size
44
45 print(f"Original_size: {original_size} bytes")
46 print(f"Compressed_size: {compressed_size} tuples")
47 print(f"Compression_ratio: {compression_ratio:.2f}:1")
48 print(f"First 10 encoded values: {encoded[:10]}")
49
50 # Verify correctness
51 if np.array_equal(simple_img, decoded):
52     print("\nDecoding successful! Images are identical.")
53 else:
54     print("\nDecoding error!")
55
56 # Display
57 fig, axes = plt.subplots(1, 2, figsize=(10, 5))
58 axes[0].imshow(simple_img, cmap='gray')
59 axes[0].set_title('Original Image')
60 axes[1].imshow(decoded, cmap='gray')
61 axes[1].set_title('Decoded Image')
62
63 for ax in axes:
64     ax.axis('off')
65
66 plt.tight_layout()
67 plt.show()

```

Listing 20: Run-Length Encoding

5.6 Assignment

1. Implement adaptive filters that adjust based on local noise variance.
2. Compare Huffman coding efficiency on images with different characteristics.
3. Implement a simple lossy compression scheme using DCT and quantization.

6 Practical Session 6: Morphological Image Processing (1 hour)

6.1 Objectives

- Understand binary image operations
- Implement dilation and erosion
- Apply opening and closing operations

6.2 Exercise 6.1: Basic Morphological Operations (30 min)

Task

Implement dilation and erosion operations.

```

1 from scipy.ndimage import binary_dilation, binary_erosion
2
3 def create_structuring_element(shape, size):
4     """Create structuring element"""
5     if shape == 'square':
6         return np.ones((size, size), dtype=np.uint8)
7     elif shape == 'cross':
8         se = np.zeros((size, size), dtype=np.uint8)
9         mid = size // 2
10        se[mid, :] = 1
11        se[:, mid] = 1
12        return se
13    elif shape == 'circle':
14        y, x = np.ogrid[-size//2:size//2+1, -size//2:size//2+1]
15        se = (x**2 + y**2 <= (size//2)**2).astype(np.uint8)
16        return se
17    else:
18        return np.ones((size, size), dtype=np.uint8)
19
20 def morphological_dilation(img, se):
21     """Perform morphological dilation"""
22     return binary_dilation(img, structure=se).astype(np.uint8) *
23         255
24
25 def morphological_erosion(img, se):
26     """Perform morphological erosion"""
27     return binary_erosion(img, structure=se).astype(np.uint8) *
28         255
29
30 def morphological_opening(img, se):
31     """Perform morphological opening (erosion followed by
    dilation)"""
32     eroded = morphological_erosion(img, se)
33     opened = morphological_dilation(eroded, se)

```

```

32     return opened
33
34 def morphological_closing(img, se):
35     """Perform morphological closing (dilation followed by
36         erosion)"""
37     dilated = morphological_dilation(img, se)
38     closed = morphological_erosion(dilated, se)
39     return closed
40
41 # Create binary image
42 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
43 _, binary_img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
44
45 # Create structuring elements
46 se_square = create_structuring_element('square', 5)
47 se_cross = create_structuring_element('cross', 5)
48 se_circle = create_structuring_element('circle', 5)
49
50 # Apply operations with square SE
51 dilated = morphological_dilation(binary_img, se_square)
52 eroded = morphological_erosion(binary_img, se_square)
53 opened = morphological_opening(binary_img, se_square)
54 closed = morphological_closing(binary_img, se_square)
55
56 # Display results
57 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
58
59 axes[0, 0].imshow(binary_img, cmap='gray')
60 axes[0, 0].set_title('Original Binary Image')
61 axes[0, 1].imshow(dilated, cmap='gray')
62 axes[0, 1].set_title('Dilation')
63 axes[0, 2].imshow(eroded, cmap='gray')
64 axes[0, 2].set_title('Erosion')
65 axes[1, 0].imshow(opened, cmap='gray')
66 axes[1, 0].set_title('Opening')
67 axes[1, 1].imshow(closed, cmap='gray')
68 axes[1, 1].set_title('Closing')
69
70 # Display structuring elements
71 axes[1, 2].imshow(se_square, cmap='gray')
72 axes[1, 2].set_title('Square SE (5x5)')
73
74 for ax in axes.flat:
75     ax.axis('off')
76
77 plt.tight_layout()
78 plt.show()
79
80 # Compare different structuring elements
81 fig, axes = plt.subplots(2, 3, figsize=(15, 10))

```

```

82 dil_square = morphological_dilation(binary_img, se_square)
83 dil_cross = morphological_dilation(binary_img, se_cross)
84 dil_circle = morphological_dilation(binary_img, se_circle)
85
86 ero_square = morphological_erosion(binary_img, se_square)
87 ero_cross = morphological_erosion(binary_img, se_cross)
88 ero_circle = morphological_erosion(binary_img, se_circle)
89
90 axes[0, 0].imshow(dil_square, cmap='gray')
91 axes[0, 0].set_title('Dilation□(Square)')
92 axes[0, 1].imshow(dil_cross, cmap='gray')
93 axes[0, 1].set_title('Dilation□(Cross)')
94 axes[0, 2].imshow(dil_circle, cmap='gray')
95 axes[0, 2].set_title('Dilation□(Circle)')
96
97 axes[1, 0].imshow(ero_square, cmap='gray')
98 axes[1, 0].set_title('Erosion□(Square)')
99 axes[1, 1].imshow(ero_cross, cmap='gray')
100 axes[1, 1].set_title('Erosion□(Cross)')
101 axes[1, 2].imshow(ero_circle, cmap='gray')
102 axes[1, 2].set_title('Erosion□(Circle)')
103
104 for ax in axes.flat:
105     ax.axis('off')
106
107 plt.tight_layout()
108 plt.show()

```

Listing 21: Morphological operations

6.3 Exercise 6.2: Advanced Morphological Operations (30 min)

Task

Implement morphological gradient, top-hat, and black-hat transforms.

```

1 def morphological_gradient(img, se):
2     """Compute□morphological□gradient"""
3     dilated = morphological_dilation(img, se)
4     eroded = morphological_erosion(img, se)
5     return dilated - eroded
6
7 def white_tophat(img, se):
8     """Compute□white□top-hat□transform"""
9     opened = morphological_opening(img, se)
10    return img - opened
11
12 def black_tophat(img, se):
13    """Compute□black□top-hat□transform"""
14    closed = morphological_closing(img, se)
15    return closed - img

```

```

16
17 def boundary_extraction(img, se):
18     """Extract boundary using erosion"""
19     eroded = morphological_erosion(img, se)
20     return img - eroded
21
22 def hole_filling(img):
23     """Fill holes in binary image"""
24     # Complement the image
25     img_comp = 255 - img
26
27     # Create marker (all zeros except border)
28     marker = np.zeros_like(img)
29     marker[0, :] = img_comp[0, :]
30     marker[-1, :] = img_comp[-1, :]
31     marker[:, 0] = img_comp[:, 0]
32     marker[:, -1] = img_comp[:, -1]
33
34     # Structuring element
35     se = np.ones((3, 3), dtype=np.uint8)
36
37     # Iterative dilation
38     prev_marker = np.zeros_like(marker)
39     while not np.array_equal(marker, prev_marker):
40         prev_marker = marker.copy()
41         marker = morphological_dilation(marker, se)
42         marker = np.minimum(marker, img_comp)
43
44     # Get filled image
45     filled = 255 - marker
46     return filled
47
48 # Create binary image
49 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
50 _, binary_img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
51
52 # Create structuring element
53 se = create_structuring_element('square', 5)
54
55 # Apply operations
56 gradient = morphological_gradient(binary_img, se)
57 w_tophat = white_tophat(binary_img, se)
58 b_tophat = black_tophat(binary_img, se)
59 boundary = boundary_extraction(binary_img, se)
60 filled = hole_filling(binary_img)
61
62 # Display results
63 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
64
65 axes[0, 0].imshow(binary_img, cmap='gray')
66 axes[0, 0].set_title('Original')

```

```
67 axes[0, 1].imshow(gradient, cmap='gray')
68 axes[0, 1].set_title('Morphological_Gradient')
69 axes[0, 2].imshow(w_tophat, cmap='gray')
70 axes[0, 2].set_title('White_Top-Hat')
71 axes[1, 0].imshow(b_tophat, cmap='gray')
72 axes[1, 0].set_title('Black_Top-Hat')
73 axes[1, 1].imshow(boundary, cmap='gray')
74 axes[1, 1].set_title('Boundary_Extraction')
75 axes[1, 2].imshow(filled, cmap='gray')
76 axes[1, 2].set_title('Hole_Filling')
77
78 for ax in axes.flat:
79     ax.axis('off')
80
81 plt.tight_layout()
82 plt.show()
```

Listing 22: Advanced morphological operations

6.4 Assignment

1. Implement hit-or-miss transform for template matching.
2. Apply morphological operations for noise removal in binary images.
3. Implement morphological reconstruction.

7 Practical Session 7: Image Segmentation (1.5 hours)

7.1 Objectives

- Implement edge detection algorithms
- Apply thresholding techniques
- Perform region-based segmentation

7.2 Exercise 7.1: Edge Detection (25 min)

Task

Implement various edge detection methods including Canny edge detector.

```

1 def canny_edge_detection(img, low_threshold, high_threshold):
2     """Implement Canny edge detection"""
3     # Use OpenCV's Canny implementation
4     edges = cv2.Canny(img, low_threshold, high_threshold)
5     return edges
6
7 def laplacian_edge_detection(img):
8     """Laplacian edge detection"""
9     laplacian = cv2.Laplacian(img, cv2.CV_64F)
10    laplacian = np.uint8(np.absolute(laplacian))
11    return laplacian
12
13 def log_edge_detection(img, sigma=1.0):
14     """Laplacian of Gaussian edge detection"""
15     from scipy.ndimage import gaussian_laplace
16
17     log = gaussian_laplace(img.astype(float), sigma)
18     log = np.uint8(np.absolute(log) * 255 / np.max(np.absolute(
19         log)))
20     return log
21
22 # Load image
23 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
24
25 # Apply different edge detectors
26 sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
27 sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
28 sobel = np.sqrt(sobel_x**2 + sobel_y**2)
29 sobel = np.uint8(sobel * 255 / np.max(sobel))
30
31 prewitt_x = apply_filter(img, np.array([[ -1, 0, 1],
32                                         [ -1, 0, 1],
33                                         [ -1, 0, 1]]))
34 prewitt_y = apply_filter(img, np.array([[ -1, -1, -1],
35                                         [ 0, 0, 0],
36                                         [ 1, 1, 1]]))

```

```

35                                     [1, 1, 1]))
36 prewitt = np.sqrt(prewitt_x**2 + prewitt_y**2)
37 prewitt = np.uint8(prewitt * 255 / np.max(prewitt))
38
39 laplacian = laplacian_edge_detection(img)
40 log = log_edge_detection(img, sigma=2.0)
41 canny = canny_edge_detection(img, 50, 150)
42
43 # Display results
44 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
45
46 axes[0, 0].imshow(img, cmap='gray')
47 axes[0, 0].set_title('Original Image')
48 axes[0, 1].imshow(sobel, cmap='gray')
49 axes[0, 1].set_title('Sobel Edge Detection')
50 axes[0, 2].imshow(prewitt, cmap='gray')
51 axes[0, 2].set_title('Prewitt Edge Detection')
52 axes[1, 0].imshow(laplacian, cmap='gray')
53 axes[1, 0].set_title('Laplacian Edge Detection')
54 axes[1, 1].imshow(log, cmap='gray')
55 axes[1, 1].set_title('LoG Edge Detection')
56 axes[1, 2].imshow(canny, cmap='gray')
57 axes[1, 2].set_title('Canny Edge Detection')
58
59 for ax in axes.flat:
60     ax.axis('off')
61
62 plt.tight_layout()
63 plt.show()

```

Listing 23: Edge detection methods

7.3 Exercise 7.2: Thresholding Techniques (25 min)

Task

Implement global, local, and adaptive thresholding methods.

```

1 def otsu_threshold(img):
2     """Compute Otsu's threshold"""
3     hist = compute_histogram(img)
4     hist = hist / np.sum(hist) # Normalize
5
6     bins = np.arange(256)
7
8     # Calculate cumulative sums
9     fn_min = np.inf
10    thresh = -1
11
12    for i in range(256):
13        # Background

```

```

14     w0 = np.sum(hist[:i])
15     if w0 == 0:
16         continue
17
18     # Foreground
19     w1 = np.sum(hist[i:])
20     if w1 == 0:
21         break
22
23     # Means
24     mu0 = np.sum(bins[:i] * hist[:i]) / w0
25     mu1 = np.sum(bins[i:] * hist[i:]) / w1
26
27     # Variance
28     var_between = w0 * w1 * (mu0 - mu1) ** 2
29
30     if var_between > fn_min:
31         fn_min = var_between
32         thresh = i
33
34     return thresh
35
36 def adaptive_threshold(img, block_size, C):
37     """Adaptive thresholding"""
38     return cv2.adaptiveThreshold(img, 255,
39                                 cv2.ADAPTIVE_THRESH_MEAN_C,
40                                 cv2.THRESH_BINARY, block_size, C
41                                 )
42
43 def adaptive_gaussian_threshold(img, block_size, C):
44     """Adaptive Gaussian thresholding"""
45     return cv2.adaptiveThreshold(img, 255,
46                                 cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
47                                 cv2.THRESH_BINARY, block_size, C
48                                 )
49
50 # Load image
51 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
52
53 # Global thresholding
54 _, global_thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
55
56 # Otsu's thresholding
57 otsu_thresh_value = otsu_threshold(img)
58 _, otsu_thresh = cv2.threshold(img, otsu_thresh_value, 255,
59                                cv2.THRESH_BINARY)
60
61 # Adaptive thresholding
62 adaptive_mean = adaptive_threshold(img, 11, 2)
63 adaptive_gaussian = adaptive_gaussian_threshold(img, 11, 2)

```

```

62
63 # Display results
64 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
65
66 axes[0, 0].imshow(img, cmap='gray')
67 axes[0, 0].set_title('Original Image')
68 axes[0, 1].imshow(global_thresh, cmap='gray')
69 axes[0, 1].set_title('Global Threshold (T=127)')
70 axes[0, 2].imshow(otsu_thresh, cmap='gray')
71 axes[0, 2].set_title(f'Otsu's Threshold (T={otsu_thresh_value})')
72 axes[1, 0].imshow(adaptive_mean, cmap='gray')
73 axes[1, 0].set_title('Adaptive Mean Threshold')
74 axes[1, 1].imshow(adaptive_gaussian, cmap='gray')
75 axes[1, 1].set_title('Adaptive Gaussian Threshold')
76
77 # Plot histogram with Otsu's threshold
78 axes[1, 2].hist(img.flatten(), bins=256, range=(0, 256))
79 axes[1, 2].axvline(x=otsu_thresh_value, color='r', linestyle='--',
80                   ,
81                   label=f'Otsu's threshold: {otsu_thresh_value}'
82                   )
83 axes[1, 2].set_title('Histogram with Otsu Threshold')
84 axes[1, 2].legend()
85
86 for ax in axes.flat[:-1]:
87     ax.axis('off')
88
89 plt.tight_layout()
90 plt.show()

```

Listing 24: Thresholding techniques

7.4 Exercise 7.3: Region-Based Segmentation (30 min)

Task

Implement region growing and watershed segmentation algorithms.

```

1 def region_growing(img, seed, threshold=10):
2     """Region growing segmentation"""
3     h, w = img.shape
4     segmented = np.zeros_like(img, dtype=np.uint8)
5
6     # Queue for pixels to check
7     queue = [seed]
8     segmented[seed] = 255
9
10    seed_value = img[seed]
11
12    while queue:
13        x, y = queue.pop(0)

```

```

14
15     # Check 8-connected neighbors
16     for dx in [-1, 0, 1]:
17         for dy in [-1, 0, 1]:
18             if dx == 0 and dy == 0:
19                 continue
20
21             nx, ny = x + dx, y + dy
22
23             if 0 <= nx < h and 0 <= ny < w:
24                 if segmented[nx, ny] == 0:
25                     if abs(int(img[nx, ny]) - int(seed_value)
26                        ) \
27                        < threshold:
28                         segmented[nx, ny] = 255
29                         queue.append((nx, ny))
30
31     return segmented
32
33 def watershed_segmentation(img):
34     """Watershed segmentation"""
35     # Noise removal
36     kernel = np.ones((3, 3), np.uint8)
37     opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel,
38                               iterations=2)
39
40     # Sure background area
41     sure_bg = cv2.dilate(opening, kernel, iterations=3)
42
43     # Finding sure foreground area
44     dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2,
45     5)
46     _, sure_fg = cv2.threshold(dist_transform, 0.7 *
47     dist_transform.max(), 255, 0)
48
49     # Finding unknown region
50     sure_fg = np.uint8(sure_fg)
51     unknown = cv2.subtract(sure_bg, sure_fg)
52
53     # Marker labelling
54     _, markers = cv2.connectedComponents(sure_fg)
55     markers = markers + 1
56     markers[unknown == 255] = 0
57
58     # Apply watershed
59     img_color = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
60     markers = cv2.watershed(img_color, markers)
61     img_color[markers == -1] = [255, 0, 0]
62
63     return markers, img_color

```

```

63 # Load image
64 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
65
66 # Threshold image for region growing
67 _, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
68
69 # Apply region growing (example seed point)
70 seed = (img.shape[0] // 2, img.shape[1] // 2)
71 region = region_growing(img, seed, threshold=20)
72
73 # Apply watershed segmentation
74 markers, watershed_result = watershed_segmentation(binary)
75
76 # Display results
77 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
78
79 axes[0, 0].imshow(img, cmap='gray')
80 axes[0, 0].plot(seed[1], seed[0], 'r*', markersize=15)
81 axes[0, 0].set_title('Original with Seed Point')
82 axes[0, 1].imshow(region, cmap='gray')
83 axes[0, 1].set_title('Region Growing Result')
84 axes[1, 0].imshow(binary, cmap='gray')
85 axes[1, 0].set_title('Binary Image')
86 axes[1, 1].imshow(watershed_result)
87 axes[1, 1].set_title('Watershed Segmentation')
88
89 for ax in axes.flat:
90     ax.axis('off')
91
92 plt.tight_layout()
93 plt.show()

```

Listing 25: Region-based segmentation

7.5 Exercise 7.4: Hough Transform (20 min)

Task

Implement Hough transform for line detection.

```

1 def hough_line_detection(img, threshold=100):
2     """Detect lines using Hough transform"""
3     # Edge detection
4     edges = canny_edge_detection(img, 50, 150)
5
6     # Hough line detection
7     lines = cv2.HoughLines(edges, 1, np.pi / 180, threshold)
8
9     # Create image to draw lines
10    line_img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
11

```

```

12     if lines is not None:
13         for rho, theta in lines[:, 0]:
14             a = np.cos(theta)
15             b = np.sin(theta)
16             x0 = a * rho
17             y0 = b * rho
18             x1 = int(x0 + 1000 * (-b))
19             y1 = int(y0 + 1000 * (a))
20             x2 = int(x0 - 1000 * (-b))
21             y2 = int(y0 - 1000 * (a))
22
23             cv2.line(line_img, (x1, y1), (x2, y2), (0, 0, 255),
24                       2)
25
26     return edges, line_img, lines
27
28 def hough_circle_detection(img, min_radius, max_radius):
29     """Detect circles using Hough transform"""
30     circles = cv2.HoughCircles(img, cv2.HOUGH_GRADIENT, 1, 20,
31                                param1=50, param2=30,
32                                minRadius=min_radius,
33                                maxRadius=max_radius)
34
35     circle_img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
36
37     if circles is not None:
38         circles = np.uint16(np.around(circles))
39         for i in circles[0, :]:
40             # Draw outer circle
41             cv2.circle(circle_img, (i[0], i[1]), i[2],
42                        (0, 255, 0), 2)
43             # Draw center
44             cv2.circle(circle_img, (i[0], i[1]), 2,
45                        (0, 0, 255), 3)
46
47     return circle_img, circles
48
49 # Load image
50 img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
51
52 # Detect lines
53 edges, lines_detected, lines = hough_line_detection(img,
54                                                       threshold
55                                                       =100)
56
57 # Display results
58 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
59 axes[0].imshow(img, cmap='gray')
60 axes[0].set_title('Original Image')
61 axes[1].imshow(edges, cmap='gray')

```

```

61 axes[1].set_title('Edge_Detection')
62 axes[2].imshow(lines_detected)
63 axes[2].set_title(f'Hough_Lines_{len(lines)}if_lines_is_not_None
    _else_0}_{lines}')
64
65 for ax in axes:
66     ax.axis('off')
67
68 plt.tight_layout()
69 plt.show()
70
71 if lines is not None:
72     print(f"Number_of_lines_detected:{len(lines)}")
73     print("First_5_lines_(rho,_theta):")
74     for i, (rho, theta) in enumerate(lines[:5, 0]):
75         print(f"_{Line}_{i+1}:_rho={rho:.2f},_"
76               f"theta={np.degrees(theta):.2f} ")

```

Listing 26: Hough transform for line detection

7.6 Assignment

1. Implement split and merge algorithm for region-based segmentation.
2. Compare different edge detection methods on various images.

7.7 Practical Session: Python Implementation

7.7.1 1. Split and Merge Algorithm for Region-Based Segmentation

The split and merge algorithm recursively divides an image into smaller regions based on a homogeneity criterion (splitting) and then combines adjacent similar regions (merging).

Libraries used: OpenCV, NumPy, Matplotlib

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def split_region(img, x, y, w, h, threshold):
6     region = img[y:y+h, x:x+w]
7     if np.std(region) < threshold or w <= 8 or h <= 8:
8         return [(x, y, w, h)]
9     else:
10        w2, h2 = w // 2, h // 2
11        regions = []
12        regions += split_region(img, x, y, w2, h2, threshold)
13        regions += split_region(img, x+w2, y, w2, h2, threshold)
14        regions += split_region(img, x, y+h2, w2, h2, threshold)
15        regions += split_region(img, x+w2, y+h2, w2, h2,
16                                threshold)
17        return regions

```

```

18 # Read and preprocess image
19 img = cv2.imread('image.jpg', 0)
20 img = cv2.GaussianBlur(img, (5,5), 0)
21
22 regions = split_region(img, 0, 0, img.shape[1], img.shape[0],
23                       threshold=10)
24
25 # Visualize result
26 output = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
27 for (x, y, w, h) in regions:
28     cv2.rectangle(output, (x,y), (x+w, y+h), (0,255,0), 1)
29
30 plt.imshow(output, cmap='gray')
31 plt.title("Split and Merge Segmentation")
32 plt.axis('off')
33 plt.show()

```

Listing 27: Split and Merge Region-Based Segmentation

7.7.2 2. Comparison of Edge Detection Methods

In this experiment, Sobel, Prewitt, and Canny edge detection methods are applied to the same image for comparison.

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 img = cv2.imread('image.jpg', 0)
6 img = cv2.GaussianBlur(img, (5,5), 0)
7
8 # Sobel Edge Detection
9 sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
10 sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
11 sobel = np.sqrt(sobelx**2 + sobely**2)
12
13 # Prewitt Edge Detection
14 kernelx = np.array([[1,0,-1],[1,0,-1],[1,0,-1]])
15 kernely = np.array([[1,1,1],[0,0,0],[-1,-1,-1]])
16 prewittx = cv2.filter2D(img, -1, kernelx)
17 prewitty = cv2.filter2D(img, -1, kernely)
18 prewitt = prewittx + prewitty
19
20 # Canny Edge Detection
21 canny = cv2.Canny(img, 100, 200)
22
23 # Display results
24 titles = ['Original', 'Sobel', 'Prewitt', 'Canny']
25 images = [img, sobel, prewitt, canny]
26
27 plt.figure(figsize=(10,6))
28 for i in range(4):

```

```
29     plt.subplot(2,2,i+1)
30     plt.imshow(images[i], cmap='gray')
31     plt.title(titles[i])
32     plt.axis('off')
33
34 plt.tight_layout()
35 plt.show()
```

Listing 28: Comparison of Edge Detection Methods

7.7.3 Observation

- Sobel and Prewitt detect edges using gradient information but are sensitive to noise.
- Canny provides better edge localization and noise suppression.
- Split and merge segmentation is effective for homogeneous regions but computationally expensive.